KIỂM THỬ PHẦN MỀM

Tài liệu hướng dẫn thực hành

LAB 11

CI/CD + Selenium Grid + Chiến lược Kiểm thử

GitHub Actions | Docker + Selenium Grid | Test Strategy | 7 bài tập

Thông tin bài lab

Thời gian: 240 phút | Điểm số: 10 điểm

Điều kiện: Có tài khoản GitHub + đã hoàn thành Lab 9 (POM framework)

Cài đặt cần: Docker Desktop, Git, GitHub CLI, Allure Commandline

Bài 1 (1.5đ) – GitHub Actions CI/CD cơ bản

Bài 2 (1.0đ) – Matrix Strategy: Chạy song song đa browser

Bài 3 (1.0đ) – GitHub Secrets: Bảo mật credential

Bài 4 (2.0đ) – Selenium Grid với Docker

Bài 5 (1.0đ) – Allure Report nâng cao

Bài 6 (1.5đ) – Pipeline đầy đủ + Allure lên GitHub Pages

Bài 7 (2.0đ) – Test Strategy và Test Plan cho dự án thực tế

BÀI 1 GITHUB ACTIONS CI/CD CƠ BẢN (1.5 điểm)

1.1 Mục tiêu

Tạo repository GitHub và thiết lập pipeline CI/CD đầu tiên: mỗi khi push code lên GitHub, Selenium test sẽ tự động chạy mà không cần bạn làm gì thêm.

1.2 Bước chuẩn bị

1. Push framework từ Lab 9 lên GitHub repository mới đặt tên là selenium-framework.

2. Thêm file README.md mô tả ngắn về project và cách chạy local.
3. Tạo file .gitignore để tránh commit file rác lên GitHub.

Lưu ý

Nội dung .gitignore tối thiểu cần có:

```
target/
.idea/
*.iml
screenshots/
.env
*.log
```

1.3 Tạo file GitHub Actions Workflow

Tạo thư mục .github/workflows/ trong project, rồi tạo file selenium-ci.yml bên trong:

```
# .github/workflows/selenium-ci.yml
name: Selenium Test Suite
on:
  push:
    branches: [ main, develop ]
  pull_request:
    branches: [ main ]
  workflow_dispatch:    # Cho phép bấm nút chạy thủ công
jobs:
  run-tests:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4
      - name: Cài Java 17
        uses: actions/setup-java@v4
        with:
          java-version: '17'
          distribution: 'temurin'
          cache: maven    # Cache để lần sau build nhanh hơn
      - name: Chạy Selenium Tests
        run: mvn clean test -Dbrowser=chrome -Denv=dev -DsuiteXmlFile=testng-
smoke.xml
        env:
```

```
        APP_PASSWORD: ${{ secrets.SAUCEDEMO_PASSWORD }}
      - name: Lưu kết quả test
        if: always()    # Chạy dù test pass hay fail
        uses: actions/upload-artifact@v4
        with:
          name: test-results
          path: |
            target/surefire-reports/
            target/screenshots/
          retention-days: 30
```

1.4 Bật Headless mode trong DriverFactory.java

CI server không có màn hình thật → phải bật Headless, nếu không sẽ báo lỗi 'cannot open display'.

```
public class DriverFactory {
    public static WebDriver createDriver(String browser) {
        // GitHub Actions tự đặt biến CI=true
        boolean isCI = System.getenv("CI") != null;
        return switch (browser.toLowerCase()) {
            case "firefox" -> createFirefoxDriver(isCI);
            default        -> createChromeDriver(isCI);
        };
    }

    private static WebDriver createChromeDriver(boolean headless) {
        ChromeOptions options = new ChromeOptions();
        if (headless) {
            options.addArguments("--headless=new");        // Chrome 112+
            options.addArguments("--no-sandbox");          // Bắt buộc trên Linux CI
            options.addArguments("--disable-dev-shm-usage"); // Tránh lỗi OOM
            options.addArguments("--window-size=1920,1080");
        } else {
            options.addArguments("--start-maximized");
        }
        WebDriverManager.chromedriver().setup();
        return new ChromeDriver(options);
    }

    private static WebDriver createFirefoxDriver(boolean headless) {
        FirefoxOptions options = new FirefoxOptions();
        if (headless) options.addArguments("-headless");
        WebDriverManager.firefoxdriver().setup();
```

```
        return new FirefoxDriver(options);
    }
}
```

Lỗi thường gặp

Lỗi 1: 'Chrome not found' → Ubuntu Latest đã có Chrome sẵn, dùng WebDriverManager để match version.

Lỗi 2: 'cannot open display :0' → Bạn quên bật --headless, xem lại DriverFactory.

Lỗi 3: 'Out of memory' → Thêm --disable-dev-shm-usage vào ChromeOptions.

Lỗi 4: 'Element not interactable' → CI chậm hơn local, hãy tăng explicit wait.

Lỗi 5: Test pass local nhưng fail CI → Thường do timing, kiểm tra lại implicit/explicit wait.

Tiêu chí chấm điểm

- ✓ File .github/workflows/selenium-ci.yml tồn tại trong repo
- ✓ Pipeline chạy thành công → chụp màn hình log màu xanh
- ✓ Tạo 1 test sai assertion cố ý → push → chụp màn hình log màu đỏ
- ✓ Download artifact → xem ảnh chụp màn hình của test bị fail

BÀI 2 MATRIX STRATEGY – CHẠY SONG SONG ĐA BROWSER (1.0 điểm)

2.1 Mục tiêu

Cấu hình pipeline chạy Chrome và Firefox cùng lúc thay vì lần lượt. Kết quả: thời gian CI giảm gần một nửa.

2.2 Thêm Matrix Strategy vào workflow

Sửa file selenium-ci.yml, thêm phần strategy vào job run-tests:

```
jobs:
  run-tests:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        browser: [chrome, firefox]   # Tạo 2 job chạy song song
      fail-fast: false               # Chrome fail không dừng Firefox

    steps:
      - name: Chạy test (${{ matrix.browser }})
        run: mvn clean test -Dbrowser=${{ matrix.browser }} -
DsuiteXmlFile=testng-smoke.xml

      - name: Lưu kết quả
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: test-results-${{ matrix.browser }}  # 2 artifact riêng biệt
          path: target/surefire-reports/
```

2.3 So sánh thời gian chạy

Cấu hình	Thời gian	Ghi chú
Tuần tự (không matrix)	~120 giây	Chrome xong rồi mới chạy Firefox
Song song (có matrix)	~65 giây	Chrome + Firefox chạy cùng lúc
Tiết kiệm được	~55 giây	Nhanh hơn khoảng 45%

Tiêu chí chấm điểm

- ✓ File YAML có strategy.matrix với [chrome, firefox] và fail-fast: false
- ✓ DriverFactory đã hỗ trợ Firefox headless (-headless argument)
- ✓ Chụp màn hình Actions tab cho thấy 2 job Chrome + Firefox chạy song song
- ✓ Ghi lại kết quả so sánh thời gian tuần tự vs song song

BÀI 3 GITHUB SECRETS – BẢO MẬT CREDENTIAL (1.0 điểm)

3.1 Tại sao không được hardcode password?

Hardcode password vào code rồi push lên GitHub là lỗi bảo mật nghiêm trọng. Bất kỳ ai thấy repository (kể cả public) đều đọc được password của bạn.

Lỗi thường gặp

TUYỆT ĐỐI KHÔNG làm như này:

```
String password = "secret_sauce";    // ← Lộ password lên GitHub!
String username = "standard_user";   // ← Lộ username lên GitHub!
```

3.2 Tạo GitHub Secrets – hướng dẫn từng bước

4. Vào repository trên GitHub.
5. Click tab Settings (góc phải phía trên).
6. Chọn Secrets and variables → Actions ở menu bên trái.
7. Click New repository secret.
8. Tạo 2 secret lần lượt:
   - Name: SAUCEDEMO_USERNAME | Value: standard_user
   - Name: SAUCEDEMO_PASSWORD | Value: secret_sauce

3.3 Truyền Secret vào workflow YAML

```
- name: Chạy test
  run: mvn test -Dbrowser=chrome
  env:
    APP_USERNAME: ${{ secrets.SAUCEDEMO_USERNAME }}
    APP_PASSWORD: ${{ secrets.SAUCEDEMO_PASSWORD }}
    # GitHub tự động che giá trị → log hiển thị *** thay vì giá trị thật
```

3.4 Đọc Secret trong Java

```
public String getPassword() {
    // Ưu tiên đọc từ biến môi trường (khi chạy trên CI/CD)
    String password = System.getenv("APP_PASSWORD");
    if (password == null || password.isBlank()) {
        // Fallback: đọc từ file config (khi chạy local)
        password = ConfigReader.getInstance().getProperty("app.password");
    }
    return password;
}
```

3.5 Kiểm tra bảo mật bằng lệnh grep

Chạy 2 lệnh sau để đảm bảo không có password nào bị commit vào code:

```
grep -r 'secret_sauce' src/      # Kết quả phải RỖNG
grep -r 'standard_user' src/main/  # Kết quả phải RỖNG
```

Lưu ý:
Tạo thêm file .env.example (không commit file .env thật) để developer local biết cần đặt biến gì:

```
# Sao chép file này thành .env rồi điền giá trị thật vào
APP_USERNAME=your_username_here
APP_PASSWORD=your_password_here
BASE_URL=https://www.saucedemo.com
```

Tiêu chí chấm điểm

- ✓ Đã tạo 2 GitHub Secrets: SAUCEDEMO_USERNAME và SAUCEDEMO_PASSWORD
- ✓ Workflow YAML truyền secret qua env block
- ✓ Java đọc từ System.getenv() trước, fallback về config file
- ✓ Lệnh grep không tìm thấy password nào trong src/
- ✓ Pipeline chạy thành công dù credential chỉ nằm trong Secrets

BÀI 4 SELENIUM GRID VỚI DOCKER (2.0 điểm)

Selenium Grid là gì?

Grid cho phép phân phối việc chạy test ra nhiều máy (node) khác nhau.

• Hub (Router): Trung tâm điều phối, nhận request từ test code và phân phối đến node phù hợp.

• Node: Máy thực sự chạy browser, đăng ký với Hub và thực thi test.

• Xem trạng thái tại: http://localhost:4444

Phần A – Khởi động Grid (0.5 điểm)

Bước 1: Tạo file docker-compose.yml

```
version: '3.8'
services:
  selenium-hub:
    image: selenium/hub:4.18.1
    container_name: selenium-hub
    ports:
      - "4442:4442"
      - "4443:4443"
      - "4444:4444"    # Grid UI và WebDriver endpoint
```

```
  chrome-node-1:
    image: selenium/node-chrome:4.18.1
    shm_size: 2gb            # Quan trọng: Chrome cần shared memory
    depends_on: [ selenium-hub ]
    environment:
      - SE_EVENT_BUS_HOST=selenium-hub
      - SE_NODE_MAX_SESSIONS=3
      - SE_NODE_SESSION_TIMEOUT=300
  chrome-node-2:
    image: selenium/node-chrome:4.18.1
    shm_size: 2gb
    depends_on: [ selenium-hub ]
    environment:
      - SE_EVENT_BUS_HOST=selenium-hub
      - SE_NODE_MAX_SESSIONS=3
  firefox-node:
    image: selenium/node-firefox:4.18.1
    shm_size: 2gb
    depends_on: [ selenium-hub ]
    environment:
      - SE_EVENT_BUS_HOST=selenium-hub
      - SE_NODE_MAX_SESSIONS=2
```

Bước 2: Khởi động và kiểm tra

```
docker-compose up -d       # Khởi động tất cả container ở background
docker ps                  # Kiểm tra 4 container đang chạy
# Mở trình duyệt, truy cập: http://localhost:4444
# → Chụp màn hình Grid Console UI cho thấy 3 node đã đăng ký với Hub
docker-compose down        # Tắt Grid khi xong việc
```

Phần B – Kết nối Framework với Grid (0.75 điểm)

Sửa DriverFactory.java – thêm hỗ trợ RemoteWebDriver

```
public static WebDriver createDriver(String browser) {
    String gridUrl = System.getProperty("grid.url");
    if (gridUrl != null && !gridUrl.isBlank()) {
        return createRemoteDriver(browser, gridUrl);  // Chạy trên Grid
    }
    return createLocalDriver(browser);  // Chạy local bình thường
}
private static WebDriver createRemoteDriver(String browser, String gridUrl) {
    DesiredCapabilities caps = new DesiredCapabilities();
    caps.setBrowserName(browser.toLowerCase());
    if (browser.equalsIgnoreCase("chrome")) {
        ChromeOptions options = new ChromeOptions();
        options.addArguments("--no-sandbox", "--disable-dev-shm-usage");
        caps.merge(options);
    }
    try {
        URL gridEndpoint = new URL(gridUrl + "/wd/hub");
        RemoteWebDriver driver = new RemoteWebDriver(gridEndpoint, caps);
        driver.manage().timeouts().implicitlyWait(Duration.ofSeconds(10));
        return driver;
    } catch (MalformedURLException e) {
        throw new RuntimeException("Grid URL không hợp lệ: " + gridUrl);
    }
}
```

Tạo file testng-grid.xml – chạy song song 4 luồng

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE suite SYSTEM "http://testng.org/testng-1.0.dtd">
<suite name="Grid Test Suite" parallel="tests" thread-count="4">
  <test name="Chrome – LoginTest">
    <parameter name="browser" value="chrome"/>
    <classes><class name="tests.LoginTest"/></classes>
  </test>
  <test name="Chrome – CartTest">
    <parameter name="browser" value="chrome"/>
    <classes><class name="tests.CartTest"/></classes>
  </test>
  <test name="Firefox – LoginTest">
    <parameter name="browser" value="firefox"/>
    <classes><class name="tests.LoginTest"/></classes>
  </test>
  <test name="Firefox – CartTest">
    <parameter name="browser" value="firefox"/>
    <classes><class name="tests.CartTest"/></classes>
  </test>
</suite>
```

```
# Lệnh chạy test với Grid:
mvn test -Dgrid.url=http://localhost:4444 -DsuiteXmlFile=testng-grid.xml
# Trong lúc chạy → mở http://localhost:4444 → chụp màn hình nhiều session
đang hoạt động
```

Phần C – Đo hiệu suất (0.75 điểm)

Chuẩn bị 8 test method, đo thời gian chạy và điền vào bảng:

Cấu hình	Số thread	Thời gian chạy	Hệ số tăng tốc
Tuần tự (local)	1	... giây	1.0x (baseline)
Song song Grid – 2 thread	2	... giây	... x
Song song Grid – 4 thread	4	... giây	... x

Tiêu chí chấm điểm

- ✓ docker-compose.yml có 1 Hub + 2 Chrome Node + 1 Firefox Node
- ✓ Chụp màn hình Grid Console UI: 3 node đã đăng ký
- ✓ DriverFactory hỗ trợ RemoteWebDriver khi có -Dgrid.url
- ✓ Chụp màn hình nhiều session chạy đồng thời trên Grid Console
- ✓ Điền đầy đủ bảng so sánh thời gian (3 hàng: 1 / 2 / 4 thread)

BÀI 5 ALLURE REPORT NÂNG CAO VỚI ANNOTATION (1.0 điểm)

5.1 Thêm Allure vào pom.xml

```
<properties>
  <allure.version>2.26.0</allure.version>
</properties>
<dependencies>
  <dependency>
    <groupId>io.qameta.allure</groupId>
    <artifactId>allure-testng</artifactId>
    <version>${allure.version}</version>
  </dependency>
</dependencies>
<build>
```

```
  <plugins>
    <plugin>
      <groupId>io.qameta.allure</groupId>
      <artifactId>allure-maven</artifactId>
      <version>2.12.0</version>
    </plugin>
  </plugins>
</build>
```

5.2 Thêm Annotation vào test class

```
@Test
@Feature("Đăng nhập hệ thống")
@Story("UC-001: Đăng nhập bằng tài khoản hợp lệ")
@Description("Kiểm thử đăng nhập với username/password hợp lệ")
@Severity(SeverityLevel.CRITICAL)
public void testLoginSuccess() {
    Allure.step("Mở trang đăng nhập", () -> driver.get(BASE_URL));
    Allure.step("Nhập thông tin đăng nhập", () -> {
        loginPage.enterUsername("standard_user");
        loginPage.enterPassword("secret_sauce");
    });
    Allure.step("Click nút Đăng nhập", () -> loginPage.clickLoginButton());
    Allure.step("Kiểm tra chuyển trang thành công", () ->
        Assert.assertTrue(driver.getCurrentUrl().contains("inventory")));
}
```

5.3 Đính kèm ảnh chụp khi test FAIL

```
// Thêm vào BaseTest.java
@AfterMethod
public void tearDown(ITestResult result) {
    if (result.getStatus() == ITestResult.FAILURE) {
        attachScreenshot(driver);
    }
    driver.quit();
}
@Attachment(value = "Ảnh chụp khi thất bại", type = "image/png")
public byte[] attachScreenshot(WebDriver driver) {
    return ((TakesScreenshot) driver).getScreenshotAs(OutputType.BYTES);
}
```

5.4 Chạy và xem Allure Report

```
mvn clean test        # Chạy test, tạo thư mục target/allure-results/
mvn allure:serve      # Tự động mở báo cáo trong trình duyệt
# Hoặc tạo file HTML tĩnh:
mvn allure:report     # Tạo tại target/site/allure-maven-plugin/index.html
```

Tiêu chí chấm điểm

- ✓ pom.xml có allure-testng dependency và allure-maven plugin
- ✓ LoginTest và CartTest có @Feature, @Story, @Severity, @Description
- ✓ Mỗi test có Allure.step() ghi lại từng bước thực hiện
- ✓ BaseTest có @AfterMethod đính kèm ảnh PNG khi test fail
- ✓ Chụp màn hình Allure Report: biểu đồ pass/fail, step-by-step, ảnh khi fail

BÀI 6 PIPELINE ĐẦY ĐỦ + ALLURE LÊN GITHUB PAGES (1.5 điểm)

6.1 Tạo file selenium-full.yml

```
# .github/workflows/selenium-full.yml
name: Full Selenium CI Pipeline
on:
  push:
    branches: [ main ]
  schedule:
    - cron: '0 2 * * 1-5'   # 2:00 AM thứ 2 đến thứ 6
jobs:
  test:
    runs-on: ubuntu-latest
    strategy:
      matrix:
        browser: [chrome, firefox]
      fail-fast: false
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { java-version: '17', distribution: 'temurin', cache: maven }
      - name: Chạy test
        run: mvn clean test -Dbrowser=${{ matrix.browser }} -
DsuiteXmlFile=testng-smoke.xml
        env:
          APP_USERNAME: ${{ secrets.SAUCEDEMO_USERNAME }}
          APP_PASSWORD: ${{ secrets.SAUCEDEMO_PASSWORD }}
      - name: Upload Allure results
        if: always()
        uses: actions/upload-artifact@v4
        with:
```

```
        name: allure-results-${{ matrix.browser }}
        path: target/allure-results/
  publish-report:
    needs: test       # Chạy sau khi tất cả job test xong
    if: always()      # Kể cả khi test fail
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - name: Download kết quả Chrome
        uses: actions/download-artifact@v4
        with: { name: allure-results-chrome, path: allure-results/ }
      - name: Download kết quả Firefox
        uses: actions/download-artifact@v4
        with: { name: allure-results-firefox, path: allure-results/ }
      - name: Tạo và Publish Allure Report
        uses: simple-elf/allure-report-action@master
        with:
          allure_results: allure-results
          gh_pages: gh-pages
          allure_report: allure-report
```

6.2 Bật GitHub Pages – hướng dẫn từng bước

9. Vào Settings của repository.
10. Chọn Pages ở menu bên trái.
11. Source → Deploy from a branch.
12. Branch → gh-pages → Save.

13.

Lưu ý

Sau khi pipeline chạy xong, Allure Report sẽ xuất hiện tại:

https://{username}.github.io/{tên-repo}/

Ví dụ: https://nguyenvana.github.io/selenium-framework/

6.3 Thêm Badge vào README.md

```
[![Test Status](https://github.com/{user}/{repo}/actions/workflows/selenium-full.yml/badge.svg)]
(https://github.com/{user}/{repo}/actions)
[![Allure Report](https://img.shields.io/badge/Allure-Report-orange)]
(https://{user}.github.io/{repo}/)
```

Tiêu chí chấm điểm

- ✓ File selenium-full.yml có cả 2 job: test (matrix) + publish-report
- ✓ GitHub Pages đã bật, Allure Report xuất hiện tại link GitHub Pages
- ✓ Chụp màn hình link GitHub Pages đang hoạt động với Allure Report
- ✓ README.md có 2 badge: Test Status + Allure Report

BÀI 7 SOẠN TEST STRATEGY VÀ TEST PLAN THỰC TẾ (2.0 điểm)

Bối cảnh dự án ShopEasy – bạn đóng vai QA Lead

Dự án: Ứng dụng mua sắm online – ShopEasy

Tech stack: Java Spring Boot (API) + React (Web) + React Native (Mobile)

Team: 4 Developer (2 backend, 2 frontend), 1 QA (bạn), 1 Designer, 1 PM

Sprint: 2 tuần | Release: Mỗi 4 tuần lên production

Môi trường: Dev (localhost) → Staging (staging.shopeasy.vn) → Production (shopeasy.vn)

Sprint 5 goal: Ra mắt tính năng 'Thanh toán trả góp qua VPBank'

PHẦN A – Test Strategy Document (1.0 điểm)

Tối thiểu 600 từ, trình bày theo 5 phần dưới đây:

1. Phạm vi kiểm thử (Scope)

Loại	Module / Tính năng	Lý do
IN SCOPE	Đăng ký tài khoản	Core feature, ảnh hưởng người dùng mới
IN SCOPE	Đăng nhập / Xác thực	Bảo mật, rủi ro cao
IN SCOPE	Tìm kiếm sản phẩm	Tính năng chính, dùng hàng ngày
IN SCOPE	Giỏ hàng	Tiến hành mua sắm, ảnh hưởng doanh thu
IN SCOPE	Thanh toán	Liên quan tiền thật, rủi ro cao nhất
OUT SCOPE	Admin Dashboard	Phase 2, chưa có trong Sprint này
OUT SCOPE	Báo cáo thống kê	Phase 2, không ảnh hưởng end-user

2. Phân loại test và tỉ lệ

Loại test	Tỉ lệ	Công cụ	Lý do chọn
Unit Test	20%	JUnit 5 + Mockito	Developer tự test, nhanh, rẻ
API Test	45%	RestAssured	Thương mại điện tử phụ thuộc API rất nhiều
UI Test (Selenium)	20%	Selenium + POM	Kiểm tra trải nghiệm người dùng
Performance Test	10%	JMeter	Hệ thống phải chịu tải cao ngày sale
Security Test	5%	OWASP ZAP	Có dữ liệu thẻ tín dụng, bắt buộc phải test

3. Definition of Done – Khi nào thì "đã test xong"?

- Smoke test: 100% PASS
- Regression test: ≥ 95% PASS
- Không có bug P1 (Blocker) nào đang mở
- Không có bug P2 (Critical) nào chưa có kế hoạch xử lý
- Code coverage ≥ 80% (đo bằng JaCoCo)
- Allure Report đã được team xem xét và xác nhận

4. Quản lý rủi ro

Rủi ro	Xác suất	Tác động	Kế hoạch giảm thiểu
Sandbox VPBank không ổn định	Cao	Không test được thanh toán	Liên hệ VPBank sớm, có mock API dự phòng
Staging data bị xóa đột xuất	Trung bình	Phải tạo lại test data	Script tự động tạo test data trước mỗi lần test
API 3rd party bị down	Trung bình	Test E2E bị block	Mock service cho test offline
CI server hết disk space	Thấp	Pipeline không chạy được	Tự động xóa artifact cũ hàng tuần

5. Lịch trình kiểm thử

Loại test	Khi nào chạy	Thời gian	Trigger
Smoke Test	Sau mỗi commit	~5 phút	Tự động – GitHub Actions
Regression Test	Hàng đêm 2:00 AM	~45 phút	Cron schedule
Performance Test	Mỗi tuần (Chủ nhật)	~2 giờ	Thủ công hoặc cron
Security Scan	Trước mỗi release	~3 giờ	Thủ công trước release
UAT	Cuối mỗi Sprint	2–3 ngày	Thủ công, có PO tham gia

PHẦN B – Test Plan cho Sprint 5 (1.0 điểm)

Tính năng: Thanh toán trả góp VPBank (3/6/12 tháng, phí 0% với đơn hàng ≥ 3 triệu)

Phân tích rủi ro nghiệp vụ – 5 kịch bản có thể gây mất tiền người dùng

14. Đơn < 3 triệu nhưng hệ thống vẫn cho phép trả góp → người dùng bị thu phí ngoài ý muốn.
15. Tính số tiền trả mỗi tháng sai → người dùng bị trừ tiền không đúng.
16. Thanh toán thành công nhưng đơn hàng không được xác nhận → mất tiền, không có hàng.
17. Hệ thống lỗi giữa chừng, tiền đã bị trừ nhưng chưa confirm → tiền mất, trạng thái không rõ.
18. Session hết hạn giữa quá trình thanh toán → người dùng bị đăng xuất, mất thông tin đơn hàng.

15 Test Case cho Sprint 5

TC-ID	Tiêu đề	Loại	Ưu tiên	Kết quả mong đợi
TC-001	Thanh toán trả góp 3 tháng, đơn ≥ 3tr	API	P1	201, status=APPROVED
TC-002	Bị từ chối khi đơn < 3 triệu	API	P1	400, error=ORDER_TOO_SMALL
TC-003	UI hiển thị đúng 3 tùy chọn: 3/6/12 tháng	UI	P1	3 radio button hiển thị đúng
TC-004	Tính số tiền trả/tháng chính xác	Unit	P1	Số tiền = gốc/kỳ, phí = 0
TC-005	Hiển thị tổng tiền trả góp rõ ràng	UI	P2	Hiển thị đủ: tiền gốc + phí
TC-006	Thanh toán 6 tháng thành công	API	P1	201, status=APPROVED
TC-007	Thanh toán 12 tháng thành công	API	P1	201, status=APPROVED
TC-008	Trả góp với tài khoản VPBank hợp lệ	E2E	P1	Đơn hàng được APPROVED

TC-ID	Tiêu đề	Loại	Ưu tiên	Kết quả mong đợi
TC-009	Trả góp với thẻ VPBank hết hạn	API	P1	400, error=CARD_EXPIRED
TC-010	Trả góp khi VPBank sandbox lỗi	API	P2	503, thông báo lỗi thân thiện
TC-011	Nhấn 'Hủy' giữa chừng không trừ tiền	E2E	P1	Tiền không bị trừ
TC-012	Session hết hạn giữa thanh toán	E2E	P2	Chuyển về đăng nhập, đơn lưu lại
TC-013	Không hiện trả góp với đơn < 3tr	UI	P2	Tùy chọn trả góp bị ẩn
TC-014	Email xác nhận gửi sau khi trả góp thành công	API	P2	Email gửi trong 5 phút
TC-015	Lịch sử đơn hàng hiển thị trả góp chính xác	UI	P3	Chi tiết kỳ, số tiền hiển thị đúng

Xác định Blockers – điều gì có thể ngăn kiểm thử đúng hạn?

Lỗi thường gặp

1. Môi trường sandbox VPBank chưa ready (xác suất cao)

   → Giải pháp: Yêu cầu VPBank cung cấp sandbox trước Sprint 2 tuần

2. Test data tài khoản ngân hàng chưa có

   → Giải pháp: Làm việc với BA/PM để có tài khoản test VPBank ngay đầu Sprint

3. API thanh toán chưa có tài liệu (Swagger/Postman collection)

   → Giải pháp: Yêu cầu Backend viết API contract trước khi dev xong

4. Môi trường staging chưa deploy code Sprint 5

   → Giải pháp: Chia làm 2 giai đoạn: test với mock (tuần 1) + test staging (tuần 2)

Pipeline riêng cho Sprint 5 – nhánh feature/vpbank-payment

```
# Thêm vào on: section trong selenium-full.yml
on:
  push:
    branches: [ main, feature/vpbank-payment ]
  pull_request:
    branches: [ main, feature/vpbank-payment ]
# Thêm job mới chỉ chạy khi PR nhắm vào nhánh payment
  payment-test:
    if: github.base_ref == 'feature/vpbank-payment'
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { java-version: '17', distribution: 'temurin', cache: maven }
      - name: Chạy Payment Tests
        run: mvn test -DsuiteXmlFile=testng-payment.xml
        env:
          VPBANK_API_KEY: ${{ secrets.VPBANK_API_KEY }}
          APP_USERNAME: ${{ secrets.SAUCEDEMO_USERNAME }}
          APP_PASSWORD: ${{ secrets.SAUCEDEMO_PASSWORD }}
```

Tiêu chí chấm điểm

- ✓ Test Strategy ≥ 600 từ, có đủ 5 phần: Scope, Phân loại, DoD, Rủi ro, Lịch trình
- ✓ Phân loại test có tỉ lệ % và giải thích lý do cụ thể
- ✓ Liệt kê ≥ 4 rủi ro với xác suất, tác động, kế hoạch giảm thiểu
- ✓ 5 kịch bản rủi ro nghiệp vụ liên quan đến tiền trong Sprint 5

- ✓ 15 Test Case có đầy đủ: TC-ID, Tiêu đề, Loại, Ưu tiên, Kết quả mong đợi
- ✓ Xác định Blockers có kế hoạch xử lý cụ thể
- ✓ Thêm pipeline riêng cho nhánh feature/vpbank-payment

BẢNG ĐIỂM TỔNG KẾT LAB 11

Bài	Nội dung	Điểm	Tiêu chí chính
Bài 1	GitHub Actions CI/CD cơ bản	1.5đ	Pipeline YAML chạy thành công – ảnh chụp log xanh
Bài 2	Matrix – Chạy song song đa browser	1.0đ	2 job Chrome + Firefox chạy đồng thời – ảnh chụp
Bài 3	GitHub Secrets – Bảo mật credential	1.0đ	Không có hardcode password – kiểm tra bằng grep
Bài 4	Selenium Grid với Docker	2.0đ	3 node trên Grid + session đồng thời + bảng so sánh
Bài 5	Allure Report nâng cao	1.0đ	Report có biểu đồ, step-by-step, ảnh khi fail
Bài 6	Pipeline + Allure lên GitHub Pages	1.5đ	GitHub Pages URL hoạt động + README có badge
Bài 7	Test Strategy + Test Plan thực tế	2.0đ	Strategy đủ 5 phần + 15 TC đầy đủ cột yêu cầu
TỔNG		10.0đ	

Lưu ý:

1. Làm bài theo thứ tự: Bài 1 → 2 → 3 → 4 → 5 → 6 → 7
2. Chụp màn hình ngay sau mỗi bước thành công, đừng để đến cuối mới chụp
3. Bài 7: Viết cụ thể, thực tế – tránh câu chung chung không đo được
4. Bài 4: Đảm bảo shm_size: 2gb cho mỗi node, thiếu là Chrome crash
5. Nếu bị lỗi: đọc phần Lỗi thường gặp ở Bài 1 trước khi Google

Good bye!